

CS 543 Web Services: Assignment 2

Design Benchmark: Services, Contracts and Schema for CampusEats

Task 1: System Capabilities (Verbs)

Based on the requirements brief, the primary functional capabilities (verbs) of the CampusEats system are:

- **RegisterUser:** Registers customer, merchant, or rider accounts and manages login credentials.
- **BrowseMenu:** Retrieves lists of available campus canteens and their active menu items.
- **PlaceOrder:** Creates an order ticket, calculates subtotals, fees, and tracks processing stages.
- **ProcessPayment:** Interfaces with payment systems to authorize cards/student accounts and handles transaction audits.
- **AssignRider:** Finds an available rider and coordinates order pickup and delivery logistics.
- **UpdatePrepStatus:** Allows canteen cooks to update the order state (e.g., cooking completed, ready for rider pickup).

Task 3: Service API Contracts

Here are the operations, inputs, outputs, and errors for the individual services.

1. User Service (Lead: Nandini)

Operation	Inputs	Outputs	Errors
registerUser	name, email, phone, user_type, password	user_id, created_at	EMAIL_ALREADY_EXISTS, INVALID_PHONE
authenticateUser	email, password	session_token, role	CREDENTIALS_INVALID, USER_LOCKED
getUserProfile	user_id	name, email, phone, role	USER_NOT_FOUND

2. Order Service (Lead: Nandini)

Operation	Inputs	Outputs	Errors
placeOrder	customer_id, canteen_id, items_list, dropoff_loc	order_id, status, total_price, est_time	CANTEEN_CLOSED, ITEM_OUT_OF_STOCK, PAYMENT_DECLINED
getOrderStatus	order_id	order_id, status,	ORDER_NOT_FOUND
cancelOrder	order_id, customer_id	order_id, status, refund_id	ORDER_NOT_FOUND, PREP_ALREADY_STARTED

Task 3: Service API Contracts (Continued)**3. Canteen Service (Lead: Nandini)**

Operation	Inputs	Outputs	Errors
getCanteenCatalog	canteen_id	canteen_details,	CANTEEN_NOT_FOUND
updateItemAvailability	menu_item_id, is_available	operation_status success	ITEM_NOT_FOUND, UNAUTHORIZED

4. Delivery Service (Lead: Alka)

Operation	Inputs	Outputs	Errors
scheduleDeliveryJob	order_id, canteen_id, dropoff_building, fee	job_id, initial_status	INVALID_BUILDING, ORDER_ALREADY_SCHEDULED
updateRiderLocation	rider_id, lat, lng	ack, status	RIDER_NOT_FOUND

5. Payment Service (Lead: Alka)

Operation	Inputs	Outputs	Errors
chargeCard	order_id, user_id, amount, pay_method	txn_id, status, timestamp	INSUFFICIENT_FUNDS, GATEWAY_TIMEOUT
refundPayment	txn_id, amount	refund_txn_id, status	TRANSACTION_NOT_FOUND, ALREADY_REFUNDED

Task 4: Detailed Specification of placeOrder Operation

1. Operation Details

REST API Endpoint: POST /api/v1/orders

Inputs:

- customer_id (UUID): The unique ID of the customer placing the order.
- canteen_id (UUID): The unique ID of the canteen preparing the food.
- items (List): A list of menu items to purchase. Each item includes:
 - * menu_item_id (UUID)
 - * quantity (Integer)
 - * customization_notes (String)
- dropoff_building (String): The name of the building on campus for delivery.

Outputs:

- order_id (UUID): The generated ID for the newly created order.
- order_status (String): Initialized status, set to 'placed'.
- total_price (Decimal): Total checkout price, including calculated delivery fee and tax.
- estimated_delivery_time (Timestamp): Calculated delivery time estimate.

2. Errors and Handling Rules

- **CUSTOMER_NOT_FOUND (404):** If the customer_id does not exist.
- **CANTEEN_CLOSED (400):** If the canteen is closed or operating hours are over.
- **ITEM_OUT_OF_STOCK (400):** If an ordered menu item is currently out of stock.
- **INVALID_BUILDING (400):** If the dropoff location building name is not valid.
- **PAYMENT_DECLINED (402):** If the student campus card or digital wallet payment fails.

3. Implementation Encapsulation (Hidden Details)

To keep services loosely coupled, the placeOrder interface hides several internal operations:

- **Database Tables:** Callers are unaware that placing an order inserts data into both the 'orders' and 'order_items' tables simultaneously.
- **Service Choreography:** The Order Service coordinates requests with the User Service (validation), Canteen Service (prices), and Payment Service (billing). The client only interacts with the Order Service endpoint.
- **Dispatch Queuing:** Once payment completes, the Order Service publishes a message to a queue to alert the Delivery Service to dispatch a rider, decoupling delivery logistics from order taking.

Task 6: Service Property Validation

Each of the five services was evaluated against the core properties of a service:

Service	Properties Check	Fixes / Remediation
User Service	Reachable: Yes (REST endpoint) Self-contained: Yes (Owns user tables)	None required. The service is self-contained.
Canteen Service	Reachable: Yes (REST endpoint) Self-contained: Yes (Owns canteens/menu tables)	None required. The service is self-contained.
Order Service	Reachable: Yes (REST endpoint) Self-contained: Yes (Owns orders/order_items) Consistently Coupled: Yes (Order schema) Independent: Yes	To handle network latency, Order Service should cache menu item availability locally instead of making blocking calls to Canteen Service during checkout.
Delivery Service	Reachable: Yes (REST endpoint) Self-contained: Yes (Owns delivery/rider profiles)	None required. Operates using logical order IDs to avoid table coupling.
Payment Service	Reachable: Yes (REST endpoint) Self-contained: Yes (Owns payment/refund tables) Consistently Coupled: Yes (ChargeCard API) Independent: Yes Loosely Coupled: Yes	None required. Decoupled from orders via logical reference mapping.